

Multithreading – Data Science Masters

Q1. What is multithreading in Python? Why is it used? Name the module used to handle threads in Python.

Multithreading is a technique in which multiple threads run within the same process. A thread is a lightweight unit of execution.

Multithreading is used to perform multiple tasks concurrently, especially when a program spends time waiting for input/output operations such as file handling, network requests, or other I/O tasks.

The module used to handle threads in Python is the threading module.

```
import threading

def task():
    print("Thread is running")

t = threading.Thread(target=task)
t.start()
t.join()
```

Q2. Why is the threading module used? Write the use of activeCount(), currentThread(), and enumerate().

The threading module provides classes and functions for creating and managing threads in Python.

1. activeCount()

It returns the number of Thread objects that are currently alive.

2. currentThread()

It returns the Thread object corresponding to the caller's current thread. In modern Python, `current_thread()` is preferred.

3. enumerate()

It returns a list of all Thread objects that are currently active.

```
import threading

print("Active threads:", threading.active_count())
print("Current thread:", threading.current_thread())
print("All active threads:", threading.enumerate())
```

Q3. Explain the following functions: run(), start(), join(), isAlive().

1. run()

It contains the code executed by a thread. When a Thread object is started, its `run()` method is executed.

2. start()

It starts the thread's activity and causes the thread's `run()` method to execute in a separate thread.

3. join()

It makes the calling thread wait until the specified thread finishes its execution.

4. isAlive()

It checks whether a thread is still running and returns True or False. In modern Python, the preferred method name is `is_alive()`.

```

import threading
import time

def task():
    time.sleep(1)
    print("Task completed")

t = threading.Thread(target=task)

t.start()
print("Thread alive:", t.is_alive())

t.join()
print("Thread alive after join:", t.is_alive())

```

Q4. Write a Python program to create two threads. Thread one must print the list of squares and thread two must print the list of cubes.

```

import threading

numbers = [1, 2, 3, 4, 5]

def print_squares():
    squares = [x ** 2 for x in numbers]
    print("Squares:", squares)

def print_cubes():
    cubes = [x ** 3 for x in numbers]
    print("Cubes:", cubes)

thread1 = threading.Thread(target=print_squares)
thread2 = threading.Thread(target=print_cubes)

thread1.start()
thread2.start()

thread1.join()
thread2.join()

print("Both threads completed.")

```

Q5. State advantages and disadvantages of multithreading.

Advantages:

1. Improves responsiveness of applications.
2. Allows multiple tasks to progress concurrently.
3. Useful for I/O-bound operations.
4. Can improve resource utilization.
5. Helps perform background tasks without blocking the main program.

Disadvantages:

1. Threads share resources, which can cause synchronization problems.
2. Race conditions may occur.
3. Deadlocks can occur when threads wait for each other.
4. Multithreaded programs can be harder to debug and maintain.
5. In CPython, the Global Interpreter Lock (GIL) limits true parallel execution of Python bytecode for CPU-bound tasks.

Q6. Explain deadlocks and race conditions.

Deadlock:

A deadlock occurs when two or more threads wait indefinitely for resources held by one another. As a result, none

of the threads can continue.

Example situation:

- Thread 1 holds Lock A and waits for Lock B.
 - Thread 2 holds Lock B and waits for Lock A.
- Both threads wait forever.

Race Condition:

A race condition occurs when multiple threads access shared data at the same time and the final result depends on the order in which the threads execute.

Race conditions can often be prevented by using synchronization mechanisms such as Lock.

```
import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter
    for _ in range(1000):
        with lock:
            counter += 1

t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)

t1.start()
t2.start()
t1.join()
t2.join()

print("Counter:", counter)
```